The Journal of Open Source Software

ErisML Compiler: A structure-preserving compiler from natural language to a moral intermediate representation

Andrew H. Bond[1]

1 San José State University, San José, CA, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

`erisml-compiler` is a Python compiler that maps natural-language moral material into a canonical, structure-preserving intermediate representation called ErisML. The compiler exists to operationalise a single thesis: that moral reasoning requires a structured representation **before** decision contraction. A scalar "good / bad / safe / unsafe" label discards the very dimensions that justify or defeat a candidate action — who the stakeholders are, what commitments bind them, which authorities are legitimate, who bears imposed risk. The compiler preserves this tensorial structure as a first-class object, evaluates it through a deterministic core (`DEME` — the Deterministic Ethical Modular Evaluator), records an SHA-256-anchored audit trace, and exports both training data for human-feedback loops (`RLEF`) and synthesisable Vitis HLS C++ for an FPGA silicon target.

A second contribution closes the loop between the model's **text output** and the model's **internal state**. The I-EIP Monitor (Internal / Activation / Delta lenses) registers forward hooks on a chosen subset of transformer layers, runs a calibrated probe per layer, and compares the resulting per-layer moral vectors against the text-side IR. When the lenses disagree — by direction flips, monotone layerwise drift, equivariance failure under semantics-preserving rewrites, joint-uncertainty spikes, or audit-chain corruption — the Monitor flags the input for human review. It never overrules `DEME`.

Statement of need

Most existing alignment tooling collapses to a scalar: a reward score, a safety classifier output, a guardrail pass/fail. This is defensible as an engineering interface but it discards the structure that ethics is *about*. Two cases:

- A medical professional choosing whether to break confidentiality to warn a third party of imminent harm is not navigating a one-dimensional good–bad axis; she is balancing care, fidelity, externality, autonomy, and legitimacy simultaneously, and the weights are not free parameters of preference but consequences of her institutional role. A scalar score that returns "0.74 unsafe" cannot represent this; an IR that decomposes the situation into a 10-dimensional moral state, a stakeholder graph, a commitment registry, and a verdict with structured residue can.
- A model that produces innocuous text while its internal representations encode something the head was trained to suppress is exactly the case where a scalar safety classifier fails by construction. The I-EIP Monitor is built so that the *disagreement between text and activations is the safety signal*, not the agreement.

To our knowledge no other open-source tool currently provides this combination: a structured

moral IR, a silicon-castable evaluation kernel, and a three-lens monitor over the internal state of a deployed model. Adjacent work falls into one of three categories: toolkits for *value alignment* via RLHF / DPO that ultimately reduce to scalar reward modelling (Christiano et al., 2017; Rafailov et al., 2023); probing tools for *interpretability* that surface internal state but do not connect it to a structured ethical evaluator (Belrose et al., 2023); and *constitutional AI* style frameworks that constrain model behaviour via natural-language rules but do not produce a verifiable intermediate object (Bai et al., 2022). `erisml-compiler` occupies the structural compositional gap between these — the moral IR plays the same role for ethical reasoning that an SSA-form intermediate representation plays for code generation (Cytron et al., 1991), and the I-EIP Monitor plays the same role for a deployed model that an architecture-level performance counter plays for a deployed binary.

The intended users are AI-safety researchers (especially those who need structured failure reports rather than scalar scores), ethics review boards that need auditable provenance for AI decisions, and hardware-software co-design teams investigating real-time ethical interlocks for safety-critical agents (autonomous vehicles, surgical robots, lethal-autonomy systems).

Software description

The compiler implements a 12-pass pipeline (see `docs/architecture.md`) with a tiered extractor stack:

- **Tier 1 (Geometric):** pre-parsed JSON event stream into the deterministic core. Used by the silicon target.
- **Tier 2 (Rules):** regex/grammar-driven `RuleExtractor` over natural language.
- **Tier 2.5 (Probe):** calibrated LaBSE-backed classifier head using the sqnd-probe v10.16.9 invariance methods: spectral decoupling, variational information bottleneck, multi-head GRL adversarial, confusion loss (Alemi et al., 2017; Ganin et al., 2016).
- **Tier 3 (LLM):** OpenAI-compatible chat-completion adapter (NRP Nautilus `gpt-oss`, `qwen3`, `glm-5`; local vLLM for self-hosted models), with a critic pass that compares the LLM's canonical-form choice against a deterministic second-opinion extractor and flags disagreements for human review.

The deterministic evaluator core is shared across all tiers and consists of (i) three small finite-state machines — `CommitmentFSM`, `LegitimacyFSM`, `ConsentFSM` — each implementable in three bits of state register, and (ii) an EM-DAG (Ethical Module Directed Acyclic Graph) of 10 modules: harm, rights, fairness, autonomy, legitimacy, epistemic, care, fidelity, externality, repair. The DAG is topologically sorted at compile time and evaluated in pipeline order. The `silicon/` package emits Vitis HLS C++ for this core targeting the Xilinx Alveo U55C in the NRP Coder environment.

The I-EIP Monitor (`monitor/`, `delta/`) is the activation-side complement. `ActivationSource` is an ABC with three concrete implementations: `MockActivationSource` (deterministic synthetic hidden states for CI), `HuggingFaceActivationSource` (forward hooks on `model.model.layers` for Qwen2/Qwen3/LLaMA/Mistral, `transformer.h` for GPT-2, `encoder.layer` for BERT/RoBERTa), and `RemoteAtlasActivationSource` (paramiko-driven inference on a remote GPU host with the harness baked in as a literal string for trust control). Per-layer `ActivationProbe` instances reuse the Phase-3 `ProbeHead` shape and accept Phase-3 checkpoints directly. The `IEIPMonitor` orchestrator produces a `MonitorTrace` with a SHA-256 anchor (`trace_hash()`) that extends the existing audit chain.

The Delta lens (`delta/`) supplies `compare_morals(text_mv, activation_mv)`, the BIP equivariance check `h_□(g·x) ≈ ρ_□(g)·h_□(x)` from sqnd-probe with `ρ_□ = identity` (invariance under semantics-preserving rewrites), and five named failure-mode detectors (`text_internal_mismatch`, `layerwise_drift`, `group_symmetry_break`, `probe_uncertainty_spike`, `audit_chain_break`). The Monitor's only authorised output

when any of these fires is `requires_human_review`; verdicts remain `DEME`'s job. The threat model and trust-boundary diagram are documented in `docs/i_eip_monitor.md`.

The package is distributed on PyPI as `erisml-compiler` and on GitHub under MIT license. The CLI exposes 12 subcommands including `compile`, `validate`, `rlef`, `report`, `bundle`, `calibrate`, `correct`, `diff`, `silicon-emit`, `monitor`, `delta`, and `version`.

End-to-end demonstration

To verify that the full Phase 4 pipeline runs against a real production-class model, we instantiate the I-EIP Monitor with a `HuggingFaceActivationSource` over `Qwen/Qwen2.5-7B-Instruct` (28 transformer layers, hidden dimension 3584) hosted on a dual-Quadro-GV100 workstation reachable from the host via Tailscale + paramiko. We hook every fourth layer plus the final layer (layer indices 0, 4, 8, 12, 16, 20, 24, 27) and run the monitor + delta + equivariance pipeline on the three bundled scenarios (`nazi_attic`, `medical_confidentiality`, `whistleblower`) with random (uncalibrated) per-layer probes.

The structural findings reproduce across runs. Activation norms climb monotonically through the residual stream on every scenario (e.g. `nazi_attic`: 8.8 $\rightarrow$ 398 $\rightarrow$ … $\rightarrow$ 571 at layer 24, dropping to 402 at the final layer). Trace hashes are deterministic. The BIP equivariance check ($\rho_{\square}$ = `identity`) under a lowercase rewrite fails specifically at the final layer on two of three scenarios but passes throughout on the third — consistent with the final layer being the locus of output-distribution commitment and therefore the most surface-form sensitive. With random probes the divergence and direction-break counts are noise — calibrated probes against a real moral-language corpus are deferred to a separate paper — but the structural behaviour (hook resolution, audit-chain anchoring, equivariance sensitivity localisation) is precisely what the Phase 4 design predicts.

The experiment harness is shipped at `scripts/experiments/atlas_phase4_experiment.py` and the per-scenario JSON reports at `experiments/phase4/` in the repository. The project ships three bundled examples — `nazi_attic.txt`, `medical_confidentiality.txt`, `whistleblower.txt` — that cover the hardest cases in classical normative ethics (the trolley/inquirer/Kant exception structure) and that the compiler is known to produce structurally faithful IR for end-to-end. The repository has 142 tests passing on Ubuntu Python 3.10/3.11/3.12 in GitHub Actions CI and is MIT-licensed.

Ongoing and future work

Calibrated probe checkpoints against a real moral-language corpus (rather than the synthetic dataset shipped with the calibration stack) are in preparation, as is a separate methodological paper on the I-EIP Monitor's empirical behaviour on Qwen2.5-7B-Instruct running on NRP Nautilus. Silicon hardware bring-up on the U55C target is gated by the NRP Coder bitstream pipeline; the Vitis HLS emit is exercised in CI on every push and is bit-exact verified through hardware emulation (70/70 PASS), but on-FPGA validation remains the next milestone.

Acknowledgements

Development of `erisml-compiler` was supported by computational resources provided by the National Research Platform (NRP) Nautilus cluster.

References

Alemi, A. A., Fischer, I., Dillon, J. V., & Murphy, K. (2017). Deep variational information bottleneck. *International Conference on Learning Representations*. https://arxiv.org/abs/

1612.00410

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., & others. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv Preprint arXiv:2212.08073*. https://arxiv.org/abs/2212.08073

Belrose, N., Furman, Z., Smith, L., Halawi, D., Ostrovsky, I., McKinney, L., Biderman, S., & Steinhardt, J. (2023). Eliciting latent predictions from transformers with the tuned lens. *arXiv Preprint arXiv:2303.08112*. https://arxiv.org/abs/2303.08112

Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, *30*. https://arxiv.org/abs/1706.03741

Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., & Zadeck, F. K. (1991). Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, *13*(4), 451–490. https://doi.org/10.1145/115372.115320

Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., & Lempitsky, V. (2016). Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, *17*(59), 1–35. https://jmlr.org/papers/v17/15-239.html

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, *36*. https://arxiv.org/abs/2305.18290